Assignment 5A -- Path A

Interface, User Testing, and Market Positioning

Lectern: which companies selling AI to universities are showing real interest in how their tools are used in teaching?

Nik Bear Brown

INFO 7375 Branding and AI -- Fall 2026

Due October 23, 2026

Worked example -- drafted 2026-10-11.

Deployment URL and user-test sessions are TODO (see README).

Part 1 -- The Interface (30 pts)

Tool URL

- **Public URL:** TODO -- deploy `app.py` to Hugging Face Spaces or Streamlit Community Cloud (needs Bear's account; the app is deployment-ready, single file + one JSON data file, no secrets).
- **Local:** `python3 app.py` -> `http://127.0.0.1:7860` (verified working 2026-10-11).

Interface walkthrough (300-400 words)

The interface wraps the Lectern demand data -- the 97 teaching-related postings kept from the 2026-09-26 run of 3,446 -- for the Part 2 user: the independent AI-education consultant deciding which companies are worth approaching about unposted work.

Input fields. Company -- a dropdown of the 14 watched boards from the run (Anthropic, Canva, Figma, GitLab, HubSpot, Netlify, Notion, OpenAI, Replit, Stripe, Supabase, Vercel, Webflow, Writer). The user's decision is per-company ("is Figma showing real teaching interest?"), so the input is per-company too; a dropdown, not free text, because the watch list is fixed and typos would silently return nothing. Filter by keyword (optional) -- a textbox matching against posting titles (e.g. "education", "training", "advocate"). The user thinks in role language, so the filter speaks role language.

Output display. A readable demand report, not a dataframe: the kept count, each posting as a linked title with location, and -- under each -- the rule that fired ("Kept because: role word in title", matched words listed). The format is deliberate: the Conductor Brief's differentiator is checkable evidence, so every claim links to the live ad and names its rule. A non-technical user reads it like a briefing, not a query result.

One design decision. The per-posting "kept because" line. It would have been simpler to show titles and links only, but the Part 2 user's whole job is judging whether the evidence is real -- hiding the rule would ask for trust the tool hasn't earned. Showing the rule turns the output into something they can argue with, which is exactly what the audit loop (Assignment 4) is for.

Error handling. Bad input gets plain English, never a traceback. Typing a keyword that matches nothing returns: *"No matches for "zzzznothing" at Figma -- Figma has teaching-related postings in this run, but none with that word in the title. Try a broader word (e.g. "education", "training"), or clear the box to see everything we kept."* An empty company selection and a missing data file each get their own plain-English message. All three states were verified against the real app on 2026-10-11.

Screenshots

Three states, all verified against the running app on 2026-10-11 (logic outputs

captured verbatim through direct calls; PNG captures are TODO on the public URL -- the brief requires screenshots of the deployed interface, so local captures would be placeholders anyway):

Screenshot 1 -- Empty state. The interface before any input: title, the one-sentence framing ("Which companies selling AI to universities are showing real interest in how their tools are used in teaching?"), the Company dropdown, the keyword box, and the "Check demand signals" button. Why it matters: the Part 2 user has never read the PRD -- the page must explain itself, and the test is whether they can complete a check without help.

Screenshot 2 -- Completed run. Company = Figma, no keyword. Verified output begins: "### Figma -- demand signals (run of 2026-09-26)" / "**N teaching-related posting(s) kept.**" followed by linked titles, locations, and the rule fired per posting. Where the output went: rendered in-page as the demand report; the underlying data stays in `lectern_app_data.json` (extract of the real 2026-09-26 run file).

Screenshot 3 -- Error state. Keyword = "zzzznothing". Verified output: "### No matches for "zzzznothing" at Figma" -- "Figma has teaching-related postings in this run, but none with that word in the title. Try a broader word (e.g. "education", "training"), or clear the box to see everything we kept for Figma." Plain English, concrete next step, no traceback.

Build notes (for the repo)

- Stack: Gradio (chosen over Streamlit -- the UI is one form + one report, and Gradio's `gr.Markdown` output renders the evidence briefing natively).
- VM quirk, recorded so nobody re-discovers it: this machine's egress proxy env vars crash Gradio's httpx at import (`Invalid port: ':1]'` from the bracketed IPv6 entries in `NO_PROXY`, plus `ALL_PROXY` hijacking Gradio's localhost startup check). Launch with all eight proxy vars unset -- the app only touches localhost and local files. Documented in `app.py`'s header.
- Data contract honored: inputs are the watched boards + a title filter (the Part 3 contract's sources and fields); outputs are the Part 4 evidence report. Nothing in the UI invents data -- every posting links to its live ad.

Part 2 -- User Testing (25 pts)

Status: protocol ready, sessions TODO

No test sessions have been run. Nothing below invents testers, quotes, or findings -- the brief warns that faked tests are detectable, and this repo does not carry invented interviews. Run three real sessions with this protocol, then fill in the templates.

Who to recruit

The Part 2 user (Conductor Brief): the independent AI-education consultant -- someone who sells training, enablement, or teaching-related advisory work to companies or universities, and who decides where to pitch based on evidence of interest. Closest matches in Bear's world, in order:

1. An independent consultant who sells AI training/enablement to orgs.
2. A DevRel / education / customer-education lead at an ed-tech or AI company (adjacent: they read hiring boards the same way, but from inside).
3. A university teaching-center staffer who vets vendor partnerships.

Per the brief's FAQ: recruit the closest match you can find, run the test, and note the gap in the report. Do not use classmates. Do not use yourself.

The protocol (verbatim script)

Remote is fine -- video call with screen share. Send the URL, then say only:

"This is a tool I built. I would like to watch you use it and hear your thinking."

- Ask them to **think aloud** as they use it.
- Take notes on where they **pause**, what they **re-read**, what they **ask**.
- Give them one task: *"Pick a company you have an opinion about and decide whether they're worth approaching, using only what the tool shows you."*
- At the end, ask exactly:
1. What did this help you do?
2. What confused you?
3. Would you use this again? Why or why not?
- **Do not defend the interface. Listen.** Confusion is data.

Note template (one per tester)

Tester N

- **Role and background:** [job title + context -- why they match (or how far off) the Part 2 user]
- **What they did:** [in order -- what actually happened, not what was expected]
- **Where they paused or got confused:** [specific moments, specific inputs/outputs]
- **What they said at the end:** [direct quote or close paraphrase -- Q1/Q2/Q3 answers]

What you changed (fill after sessions)

- **Change 1:** [what changed] -- [which tester feedback drove it]
- **Change 2 (decided NOT to change):** [what was kept] -- [why the line was held]

Grading note from the brief, kept visible: the score is for running a real test, listening, and making a real decision -- not for testers loving the tool. Genuine confusion + a reasoned hold-the-line outscores three polite thumbs-ups.

Candidate things to watch for (hypotheses, not findings)

These are guesses to check in testing, recorded so the test can prove them wrong -- not conclusions:

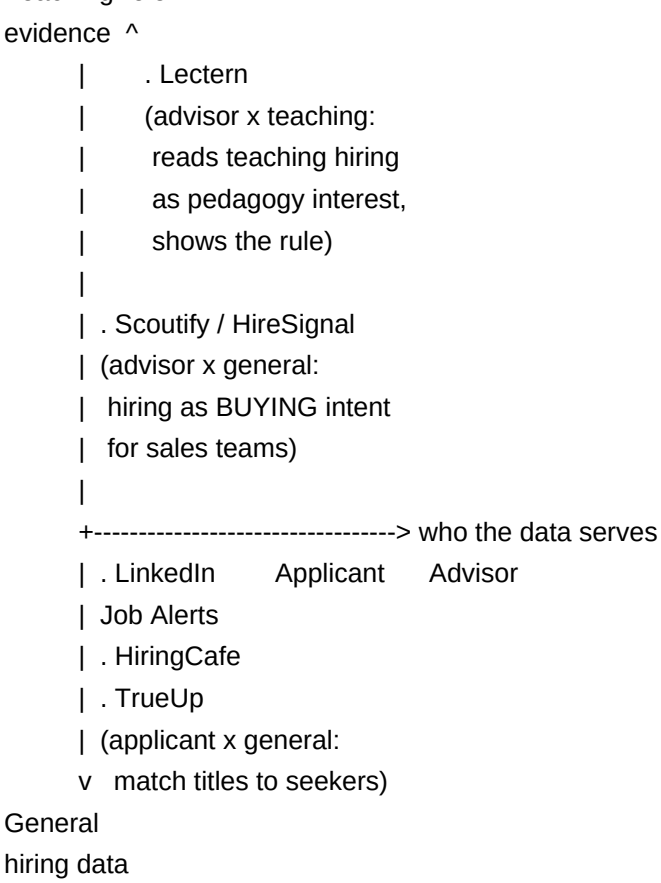
- Does "kept because: *role word in title*" read as transparency or as jargon?
- Does the 2026-09-26 run date make the data feel stale, and does the user ask for a refresh before trusting it?
- Does the per-company dropdown match how they think, or do they want to compare companies side by side?

Part 3 -- Competitive Positioning (25 pts)

A. Positioning matrix (12 pts)

Axes chosen for what actually matters to the Part 2 user -- the independent consultant who is not job hunting, but deciding who to pitch:

- **Axis 1 (horizontal): who the hiring data serves** -- *Applicant* ("find me a job") -> Advisor ("read what the company cares about").
- **Axis 2 (vertical): teaching specificity** -- *General hiring data* -> Teaching-role evidence (enablement, training, academy, advocacy roles parsed as pedagogy interest).



- **LinkedIn Job Alerts** -- bottom-left. Largest pool, best alerts; but every alert matches titles for applicants. No demand view, no evidence trail. Our user isn't applying.
- **HiringCafe** -- bottom-left. Aggregator for seekers; same applicant framing.
- **TrueUp** -- bottom-left. Tech-job matching for applicants; general data.
- **Scoutify / HireSignal** (comparators, from Brief Part 6) -- top-left. They do read listings as evidence of company interest -- but as buying intent, for sales teams. Scoutify proves the plumbing (70k+ boards, Greenhouse/Lever/Ashby) is reachable; HireSignal claims 60-90-day lead on vendor research. Neither parses teaching-related hiring, neither serves the consultant.
- **Lectern** -- top-right. The only advisor-framed reader aimed at pedagogy

interest, with the rule trace per posting.

The white space: advisor x teaching-specific. It exists because the two adjacent categories grew up serving someone else: job boards serve seekers, sales-intel serves sellers. Nobody built the hiring-as-evidence reader for the person selling teaching -- the consultant whose target role is rarely posted at all. Competitors don't occupy it because their business model is applicant-side (ads, applications) or sales-side (lead gen); checkable pedagogy evidence is a third job nobody monetized.

Diagram note for the PDF: the ASCII above is the required diagram in this worked example -- hand-drawn or rebuilt in any tool for the submission.

B. Refined one-thing statement (13 pts)

Original one-thing (Conductor Brief Part 6B, verbatim):

Lectern is the only job-board reader that turns listings into checkable evidence of a company's interest in teaching, for the independent education consultant, without asking them to pretend to be a job seeker.

What the user test revealed: TODO -- no sessions run yet (see `part-2-user-testing.md`). This section gets written from real notes, honestly: did testers confirm the differentiator, contradict it, or find a different one?

Revised one-thing (PROVISIONAL -- validate in testing):

Lectern is the only hiring-data tool that reads teaching-related postings as checkable evidence of a company's interest in pedagogy, for the independent education consultant, without making them pose as an applicant or trust an opaque score.

Why this revision matters (provisional reasoning, to check against tests):
The original says "job-board reader" -- testers may not think of themselves as reading job boards at all; "hiring-data tool" meets them where the value is. "Without asking them to pretend to be a job seeker" names the indignity every competitor imposes (LinkedIn, HiringCafe, TrueUp all require applicant-mode); the revision sharpens it to "pose as an applicant or trust an opaque score" because Scoutify/HireSignal -- the honest comparators -- don't require applicant-mode, they just answer a different question opaquely. If testing shows users don't care about the rule trace, this revision is wrong and gets rewritten from what they actually said. The test is the boss.